

Mantenimiento Preventivo

Idempotencia para evitar eventos duplicados en NestJS

Arquitectura de Software y Robustez

| 1. El Diagnóstico del Sistema



Problema Central

La arquitectura actual centraliza eventos desde el CRUD de laptops pero no protege contra reintentos duplicados.



Escenario de Fallo

Timeouts de red o reenvíos manuales generan registros repetidos en las tablas de auditoría y eventos.



Efecto Colateral

Las métricas de `/stats` se inflan, distorsionando la realidad operativa del negocio.

| Riesgos y Deuda Técnica

- ⊗ **Inestabilidad ante reintentos:** El sistema no es resiliente a fallos de comunicación básicos.
- 🗄️ **Contaminación de datos:** Integridad comprometida en las tablas de eventos y logs.
- ⚡ **Falta de Clave de Deduplicación:** Ausencia de una estrategia de clave única en la capa de servicios.
- ⚡ **Riesgo Operativo:** Decisiones de negocio basadas en métricas de eventos duplicados.

2. La Intervención

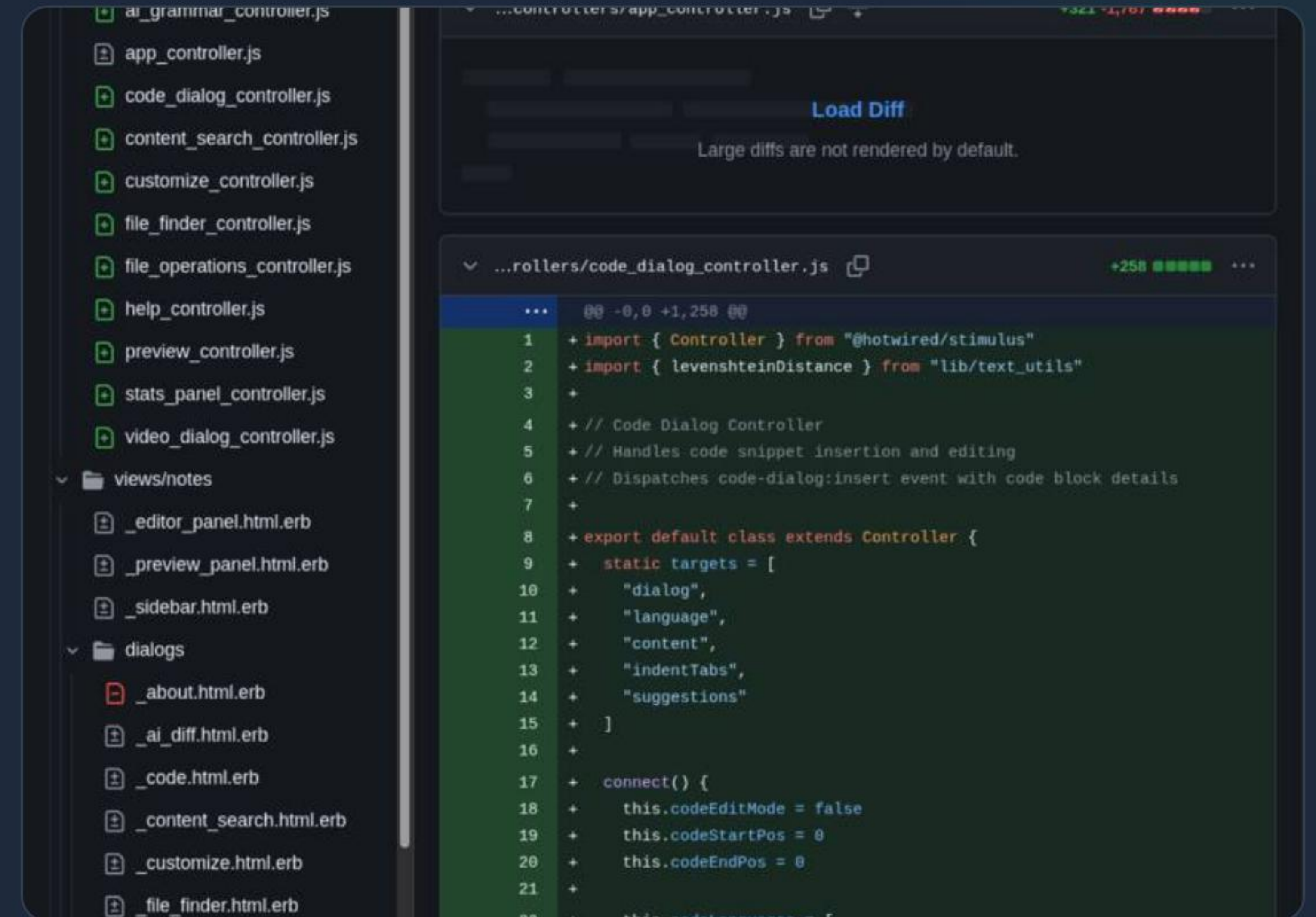
Introducción de `eventKey` como ancla de idempotencia.

| Estado Anterior (Antes)

El servicio `registerEvent` persiste cada request sin verificar si la operación ya fue procesada anteriormente.

⚠ Sin validación de duplicidad

⚠ Riesgo de doble inserción



| Modificación de Entidad y DTO

Entity: Restricción Única

```
@Column({  
  nullable: false,  
  unique: true  
})  
eventKey!: string;
```

DTO: Campo Obligatorio

```
export class CreateEventDto {  
  // ... campos previos  
  eventKey!: string;  
}
```

La base de datos ahora actúa como la última línea de defensa mediante el índice único.

| Lógica Refactorizada: EventsService

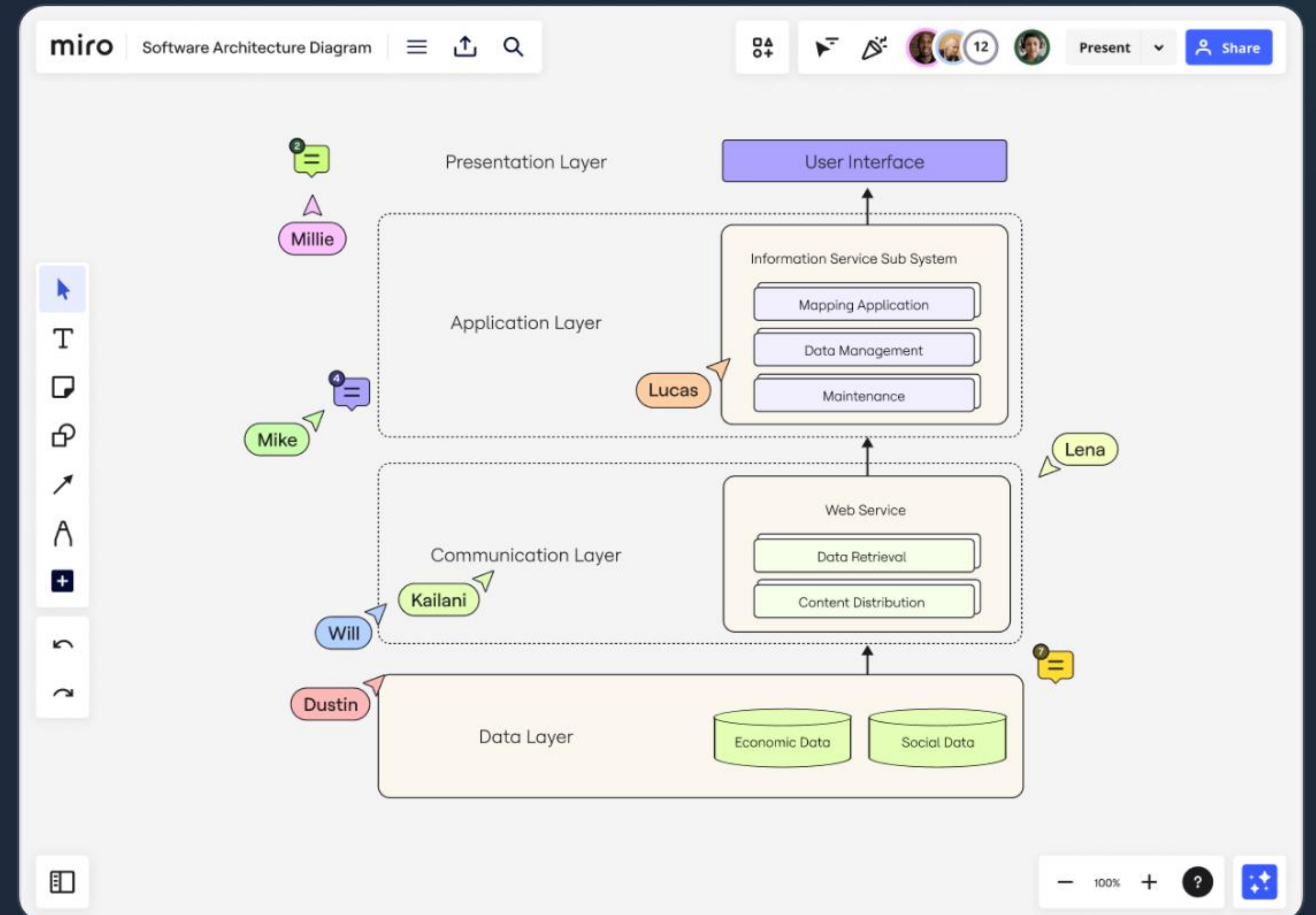
```
async registerEvent(dto: CreateEventDto) {  
  const exists = await this.createRepo.manager.count(  
    CreateEventEntity, { where: { eventKey: dto.eventKey } }  
  );  
  
  if (exists > 0) {  
    return { ok: true, duplicated: true };  
  }  
  
  const ev = this.createRepo.create({ ...dto });  
  await this.createRepo.save(ev);  
  return { ok: true };  
}
```

Implementación del patrón **Check-Then-Save** para retorno temprano en caso de duplicidad detectada.

| Integración en LaptopsService

El productor de eventos genera una llave determinística combinando contexto y ID de entidad.

```
const event = {  
  // ... data  
  eventKey: `crud:CREATE:laptop:${id}`  
};
```



| 3. La Justificación Preventiva

- 🛡️ **ISO/IEC 14764:** Define el mantenimiento preventivo como la detección y eliminación de defectos latentes.
- 🔴 **Defecto Latente:** La ausencia de idempotencia es un riesgo que solo se manifiesta bajo condiciones de red inestables.
- ✅ **Consistencia Operacional:** Garantiza que el estado final sea predecible independientemente del número de intentos.

| Impacto en la Confiabilidad

0%

Eventos Duplicados

Resultados Clave

Eliminación total del ruido en auditoría y métricas. El sistema CRUD de laptops ahora tolera fallos de red sin contaminar el historial de eventos.

| 4. Prueba de Vida

Paso	Acción	Resultado Esperado
1	POST /laptops	Laptop creada y Evento 1 guardado
2	Repetir POST	Response: duplicated: true (Sin nueva fila)
3	GET /events	Solo existe 1 registro para esa eventKey



¿Preguntas?

Implementando robustez mediante Idempotencia



| Image Sources



https://new-uploads-akitaonrails.s3.us-east-2.amazonaws.com/frankmd/2026/02/screenshot-2026-02-01_17-29-19.jpg

Source: akitaonrails.com



https://images.ctfassets.net/w6r2i5d8q73s/35z1WmRtjF6YGNv3z4xED8/23bc67d0f721255012a6cbe4ca04ecfd/software-architecture-diagram-tool_hero_xxl_sub-use-case_img_EN

Source: miro.com



https://png.pngtree.com/png-vector/20260425/ourlarge/pngtree-database-with-wireless-connection-icon-in-outline-style-vector-png-image_19165152.webp

Source: pngtree.com



<https://www.free-power-point-templates.com/articles/wp-content/uploads/2014/02/free-question-mark-powerpoint.jpg>

Source: www.free-power-point-templates.com